Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Разработка прототипа системы бронирования автомобилей на выбранные даты на базе веб-интерфейса»

Студент	ИУ7-21М		Салженикин Д.А.
	(Группа)	(Подпись, дата)	(И. О. Фамилия)
Руководитель курсовой работы			А. А. Ступников
		(Подпись, дата)	(И. О. Фамилия)
Консультант			
		(Подпись, дата)	(И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

1 Аналитическая часть **7**

1.1 Описание системы 7

1.2 Функциональные требования 7

1.3 Входные данные 9

1.4 Выходные данные 10

1.5 Состав системы 10

1.6 Требования к безопасности 11

1.7 Требования к отказоустойчивости и наблюдаемости 11

1.8 Вывод 12

2 Конструкторская часть **13**

2.1 Концептуальный дизайн 13

2.2 Сценарии функционирования системы 14

2.3 Диаграммы прецедентов 16

2.4 Архитектура микросервисов 17

2.5 Проектирование авторизации 18

2.6 Вывод 18

3 Технологическая часть **19**

3.1 Выбор операционной системы 19

3.2 Выбор СУБД 19

3.3 Выбор языка разработки и backend-фреймворка 19

3.4 Выбор фреймворка пользовательского интерфейса 20

3.5 Асинхронное взаимодействие 20

3.6 Реализация микросервисов 20

3.6.1 UI-service 21

3.6.2 Identity-service 21

3.6.3 Cars-service 21

3.6.4 Rental-service 21

3.6.5 Payment-service 22

3.6.6 Statistics-service 22

3.6.7 Gateway-service 22
3.7 Авторизация и межсервисное взаимодействие 22
3.8 Обработка ошибок и деградация 22
3.9 Логирование 23
3.10 Развертывание 23
3.11 CI/CD 24
3.12 Высокоуровневый дизайн пользовательского интерфейса 24
3.13 Вывод 30

ЗАКЛЮЧЕНИЕ **31**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **32**

ВВЕДЕНИЕ

В современном мире сервисы краткосрочной аренды автомобилей становятся одним из распространенных способов получения транспорта без необходимости владения личным автомобилем. Пользователю важно быстро выбрать подходящую машину, указать даты аренды, оформить бронирование и получить подтверждение оплаты через единый веб-интерфейс. Для администратора такой системы важны управление учетными записями, контроль событий и получение статистики по действиям пользователей.

Разработка подобной системы требует не только реализации пользовательского интерфейса, но и построения распределенной архитектуры. Отдельные компоненты должны отвечать за авторизацию, учет автомобилей, бронирование, платежи, сбор статистики и агрегацию запросов. Такой подход позволяет изолировать ответственность сервисов, упростить сопровождение и обеспечить независимое развертывание компонентов.

Особое значение в распределенной системе имеет безопасность взаимодействия. Все запросы между сервисами должны выполняться с использованием токенов, выданных Identity Provider по протоколу OpenID Connect. Для проверки подлинности токенов используется JWT и набор ключей JWKs, что позволяет сервисам валидировать запросы без обращения к сервису авторизации при каждом пользовательском действии.

Целью курсовой работы является разработка прототипа системы бронирования автомобилей на выбранные даты на базе веб-интерфейса.

Для достижения поставленной цели необходимо решить следующие задачи:

- описать предметную область и функциональные требования к системе бронирования автомобилей;
- спроектировать микросервисную архитектуру системы;
- реализовать сервис пользовательского интерфейса;
- реализовать собственный сервис авторизации и данных пользовательских аккаунтов с поддержкой OpenID Connect;

- реализовать сервисы автомобилей, бронирования, платежей, статистики и агрегирования запросов;
- организовать асинхронную передачу событий через Apache Kafka;
- обеспечить авторизацию межсервисных запросов на основе JWT-токенов;
- подготовить развертывание системы в Kubernetes с использованием Helm charts и Ingress;
- настроить сборку и развертывание сервисов через CI/CD.

1 Аналитическая часть

1.1 Описание системы

Разрабатываемая система представляет собой веб-портал для бронирования автомобилей на выбранные пользователем даты. Пользователь может зарегистрироваться, пройти авторизацию, просмотреть список автомобилей, выбрать период аренды, оформить бронирование и оплатить его. После оформления пользователь может просматривать свои бронирования, завершать активную аренду или отменять ее.

Система построена как набор микросервисов. Каждый микросервис отвечает за отдельную задачу и имеет доступ только к собственной области данных. Взаимодействие пользователя с бизнес-функциями выполняется через сервис-агрегатор, который координирует вызовы сервисов автомобилей, бронирования, платежей, статистики и авторизации.

Для авторизации используется собственный Identity Provider, реализующий OpenID Connect. Пользовательский интерфейс рассматривается как стороннее приложение, получающее токен через Authorization Code Flow. Все защищенные запросы выполняются с JWT-токеном, который валидируется сервисами по JWKs.

1.2 Функциональные требования

Система должна обеспечивать следующие функции:

1. Регистрация новых пользователей через пользовательский интерфейс.
2. Авторизация пользователей через собственный Identity Provider.
3. Разделение пользователей на роли:
   - гость;
   - пользователь;
   - администратор.
4. Просмотр списка автомобилей.
5. Фильтрация автомобилей по марке, модели, номеру и типу.

6. Выбор дат аренды автомобиля.
7. Создание бронирования автомобиля на выбранные даты.
8. Создание платежа при бронировании автомобиля.
9. Просмотр списка собственных бронирований.
10. Завершение активного бронирования.
11. Отмена бронирования.
12. Создание пользователей администратором.
13. Просмотр администратором списка пользователей.
14. Просмотр администратором статистики, собранной по событиям системы.

Функции пользователей представлены в таблице 1.1.

Таблица 1.1 – Функции пользователей

Гость	1. Просмотр страницы входа; 2. Регистрация в системе; 3. Переход к форме авторизации Identity Provider.
Пользователь	1. Авторизация в системе; 2. Просмотр списка автомобилей; 3. Поиск автомобилей по основным полям; 4. Выбор дат аренды; 5. Бронирование автомобиля; 6. Автоматическое создание платежа; 7. Просмотр собственных бронирований; 8. Завершение активной аренды; 9. Отмена бронирования.

Администратор	1. Все функции пользователя; 2. Создание новых пользователей; 3. Просмотр списка пользователей; 4. Просмотр отчета статистики; 5. Просмотр последних событий системы.

1.3 Входные данные

Входные параметры системы представлены в таблице 1.2.

Таблица 1.2 – Входные данные

Операция	**Входные данные**
Регистрация пользователя	1. *логин*; 2. *email*; 3. *имя пользователя*; 4. *пароль*.
Авторизация пользователя	1. *логин*; 2. *пароль*; 3. *client_id*; 4. *redirect_uri*; 5. *scope*: openid, profile, email.
Получение списка автомобилей	1. *номер страницы*; 2. *размер страницы*; 3. *признак отображения занятых автомобилей*; 4. *поисковая строка* в пользовательском интерфейсе.
Создание бронирования	1. *идентификатор автомобиля*; 2. *дата начала аренды*; 3. *дата окончания аренды*; 4. *JWT-токен пользователя*.
Завершение или отмена бронирования	1. *идентификатор бронирования*; 2. *JWT-токен пользователя*.

Продолжение на следующей странице

Операция	**Входные данные**
Создание пользователя администратором	1. *логин*; 2. *email*; 3. *имя пользователя*; 4. *пароль*; 5. *JWT-токен администратора.*

1.4 Выходные данные

Выходными данными системы являются веб-страницы и JSON-ответы API. Основные выходные данные представлены в таблице 1.3.

Таблица 1.3 – Выходные данные

Гость	1. Страница входа; 2. Форма регистрации; 3. Форма авторизации Identity Provider.
Пользователь	1. Список автомобилей: марка, модель, регистрационный номер, мощность, тип, цена за день, доступность; 2. Расчет стоимости аренды за выбранный период; 3. Список бронирований: автомобиль, даты, статус бронирования, статус и сумма платежа; 4. Сообщения об успешном бронировании, завершении или отмене аренды.
Администратор	1. Список пользователей: логин, email, имя, роль; 2. Отчет статистики: количество событий, количество типов действий, количество пользователей в отчете; 3. Последние события системы: время, пользователь, действие, данные события.

1.5 Состав системы

Система состоит из следующих сервисов:

- **ui-service** — сервис пользовательского интерфейса, реализующий Single Page Application на React;

- **identity-service** — сервис авторизации и данных пользовательских аккаунтов, выполняющий роль Identity Provider;
- **cars-service** — сервис автомобилей;
- **rental-service** — сервис бронирования автомобилей;
- **payment-service** — сервис платежей;
- **statistics-service** — сервис сбора и предоставления статистики;
- **gateway-service** — сервис агрегирования запросов и предоставления ограниченного API для внешних клиентов;
- **kafka** — брокер сообщений Apache Kafka для асинхронной передачи событий.

1.6 Требования к безопасности

Все методы `/api/**`, кроме публичных методов регистрации, авторизации и callback, должны быть закрыты token-based авторизацией. В качестве токена используется JWT, полученный от Identity Provider. Каждый сервис валидирует токен самостоятельно по JWKs. Пользователь определяется по claims токена, заголовок `X-User-Name` для определения пользователя не используется.

При отсутствии токена, ошибке валидации JWT или истечении срока действия токена сервис должен вернуть HTTP-статус 401.

1.7 Требования к отказоустойчивости и наблюдаемости

Система должна обрабатывать ошибки зависимых сервисов. Некритичный функционал должен деградировать без полного отказа пользовательского сценария. Например, при временной недоступности сервиса статистики основная операция бронирования не должна завершаться ошибкой.

Все сервисы должны логгировать входящие запросы и основные бизнес-события. Действия пользователя дополнительно отправляются через Kafka в сервис статистики.

1.8 Вывод

В аналитической части были определены назначение системы, роли пользователей, функциональные требования, входные и выходные данные, состав микросервисов, а также требования к безопасности и отказоустойчивости.

2 Конструкторская часть

2.1 Концептуальный дизайн

Для описания функциональной модели системы используется нотация IDEF0. На рисунке 2.1 приведена верхнеуровневая модель системы бронирования автомобилей. На рисунке 2.2 представлена декомпозиция основных процессов: регистрация и авторизация, просмотр автомобилей, бронирование, оплата и сбор статистики.

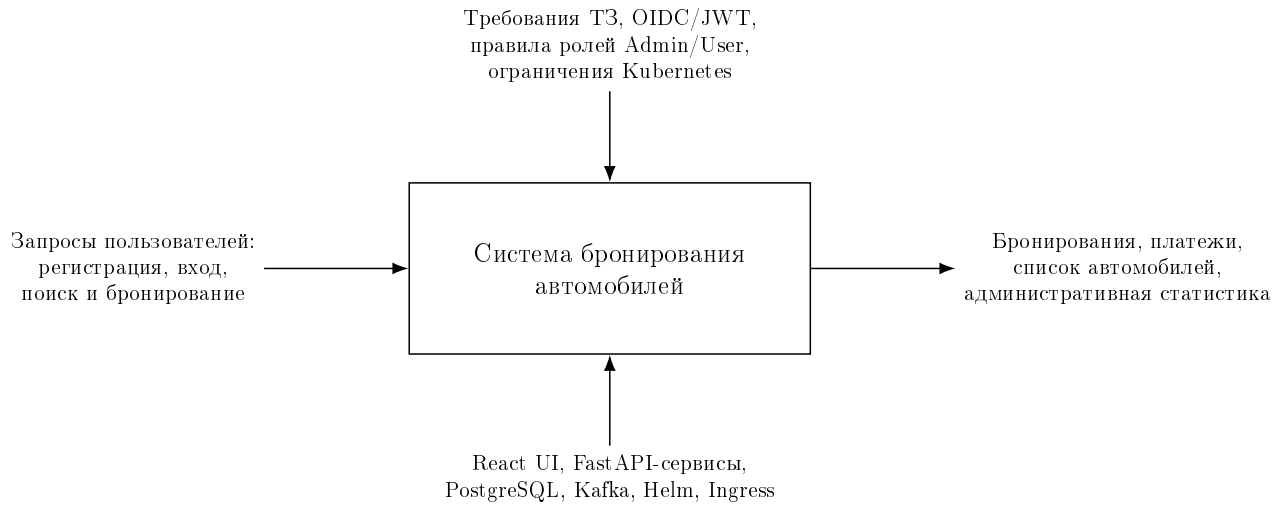


Рисунок 2.1 – Концептуальная модель системы в нотации IDEF0

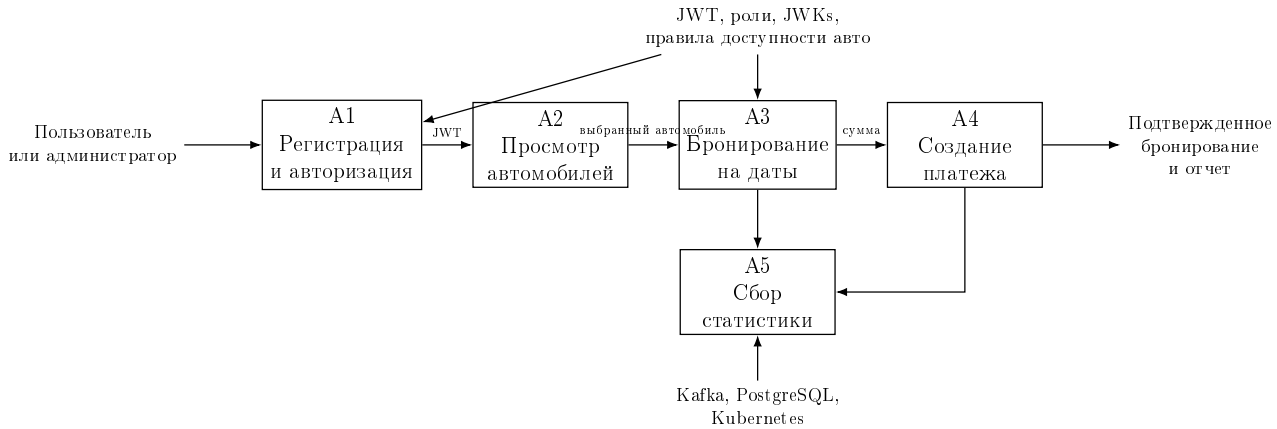


Рисунок 2.2 – Детализированная концептуальная модель системы в нотации IDEF0

2.2 Сценарии функционирования системы

Регистрация пользователя

1. Пользователь открывает веб-интерфейс и выбирает режим регистрации.
2. Пользователь вводит логин, email, имя и пароль.
3. UI отправляет запрос на регистрацию в gateway-service.
4. Gateway-service передает запрос в identity-service.
5. Identity-service проверяет уникальность логина и email, сохраняет пользователя с ролью User и возвращает результат.
6. Пользователь получает сообщение об успешном создании аккаунта и может выполнить вход через Identity Provider.

Авторизация пользователя

1. Пользователь нажимает кнопку входа в веб-интерфейсе.
2. UI формирует запрос Authorization Code Flow и перенаправляет пользователя на `/oauth/authorize`.
3. Identity Provider показывает форму входа.
4. После ввода логина и пароля Identity Provider создает authorization code и перенаправляет пользователя на callback UI.
5. UI через gateway-service обменивает authorization code на JWT.
6. JWT сохраняется в браузере и используется в последующих API-запросах.

Бронирование автомобиля

1. Пользователь выбирает даты начала и окончания аренды.
2. Пользователь выбирает доступный автомобиль и нажимает кнопку бронирования и оплаты.

3. UI отправляет запрос в gateway-service с идентификатором автомобиля и выбранными датами.
4. Gateway-service валидирует JWT и получает информацию об автомобиле из cars-service.
5. Gateway-service резервирует автомобиль, изменяя его доступность.
6. Gateway-service создает платеж в payment-service.
7. Gateway-service создает запись бронирования в rental-service.
8. Gateway-service отправляет событие о бронировании в Kafka.
9. UI отображает пользователю созданное бронирование и статус платежа.

Завершение бронирования

1. Пользователь открывает страницу своих бронирований.
2. Пользователь нажимает кнопку завершения активной аренды.
3. Gateway-service получает данные бронирования из rental-service.
4. Gateway-service освобождает автомобиль через cars-service.
5. Gateway-service меняет статус бронирования на FINISHED через rental-service.
6. Gateway-service отправляет событие в Kafka.

Отмена бронирования

1. Пользователь открывает список бронирований.
2. Пользователь нажимает кнопку отмены.
3. Gateway-service освобождает автомобиль.
4. Gateway-service меняет статус бронирования на CANCELED.
5. Gateway-service отменяет платеж или помещает операцию отмены в очередь повторной обработки при недоступности платежного сервиса.

6. Gateway-service отправляет событие об отмене в Kafka.

Просмотр статистики администратором

1. Администратор проходит авторизацию.
2. UI отображает вкладку администрирования.
3. Администратор открывает отчет статистики.
4. Gateway-service проверяет наличие роли Admin в JWT.
5. Gateway-service запрашивает данные у statistics-service.
6. UI отображает сводку и последние события.

2.3 Диаграммы прецедентов

В системе выделены 3 роли: неавторизованный пользователь, авторизованный пользователь и администратор. На рисунках 2.3-2.5 представлены диаграммы прецедентов для каждой роли.

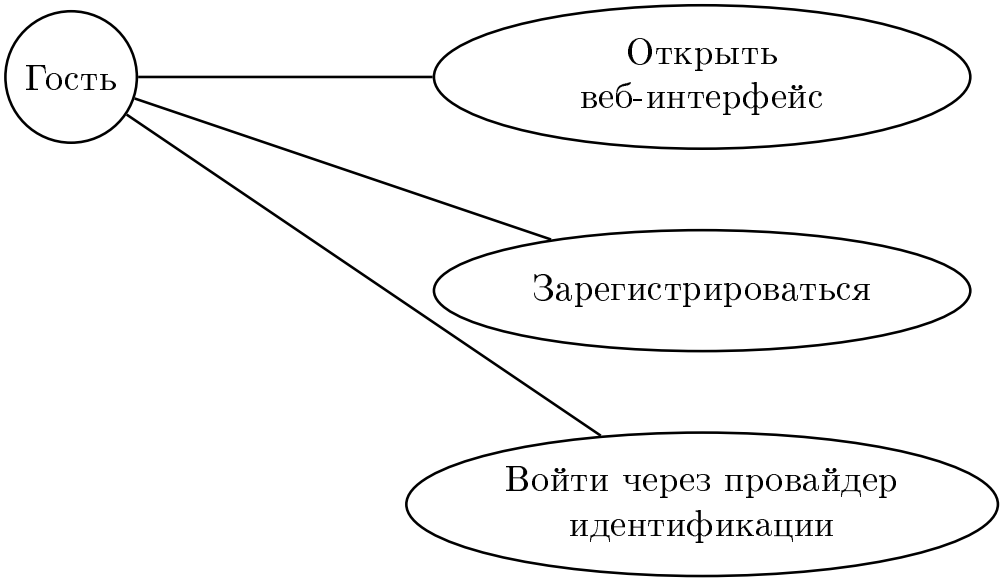


Рисунок 2.3 – Диаграмма прецедентов с точки зрения неавторизованного пользователя

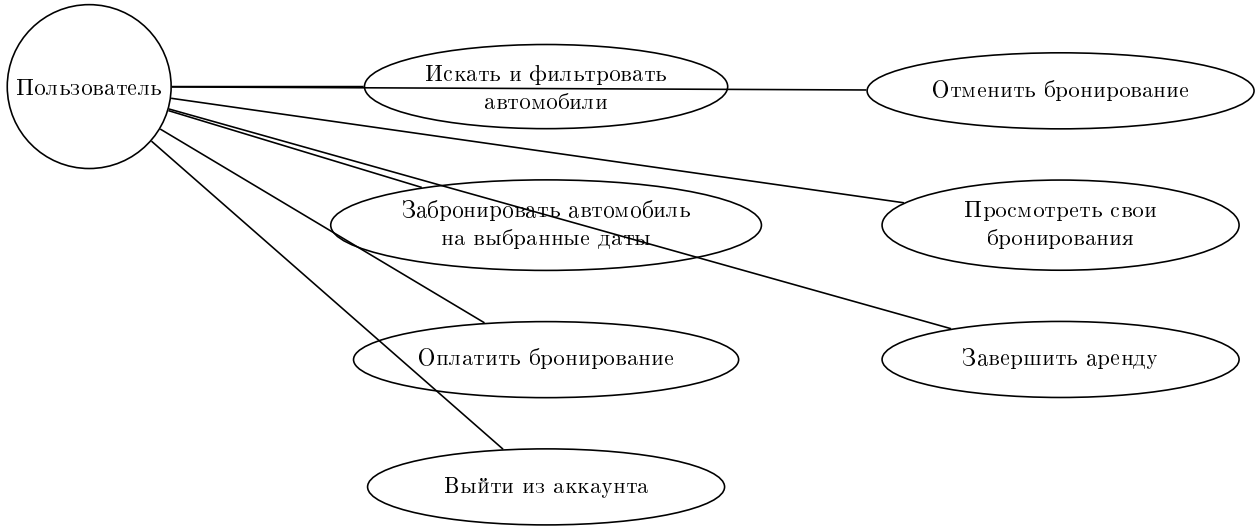


Рисунок 2.4 – Диаграмма прецедентов с точки зрения авторизованного пользователя

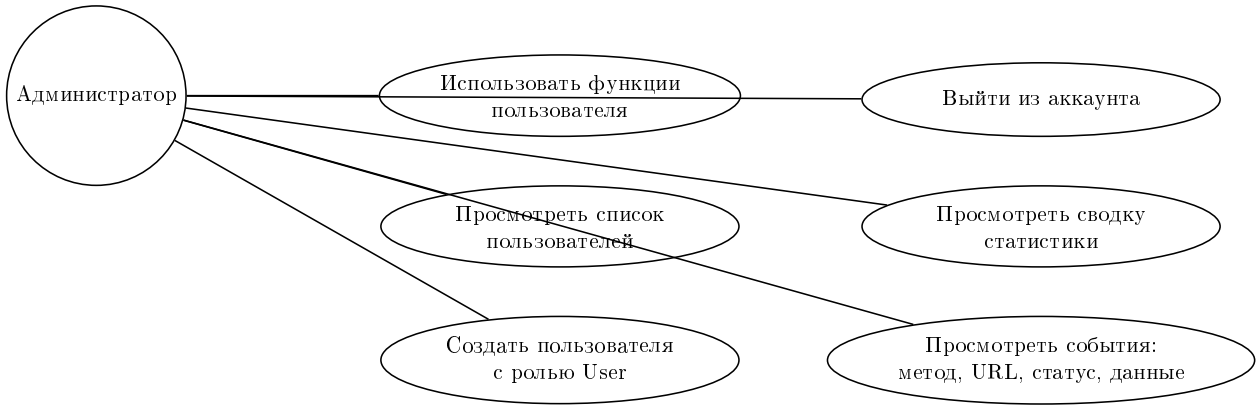


Рисунок 2.5 – Диаграмма прецедентов с точки зрения администратора

2.4 Архитектура микросервисов

В системе используется микросервисная архитектура. Клиентское приложение обращается к gateway-service, который агрегирует запросы и скрывает внутреннюю структуру системы. Внутренние сервисы не публикуются наружу напрямую.

Основной поток данных при бронировании включает cars-service, payment-service и rental-service. События пользовательских действий отправляются в Kafka, после чего statistics-service обрабатывает их и сохраняет для последующего отображения администратору.

2.5 Проектирование авторизации

Identity-service реализует следующие функции OpenID Connect:

- endpoint авторизации `/oauth/authorize`;
- endpoint обмена кода на токен `/oauth/token`;
- metadata `/.well-known/openid-configuration`;
- endpoint JWKs `/oauth/jwks`;
- scopes `openid`, `profile`, `email`.

JWT содержит сведения о пользователе, email, имени и ролях. Сервисы получают JWKs от Identity Provider и валидируют токены без дополнительного запроса к нему при каждом пользовательском действии.

2.6 Вывод

В конструкторской части были описаны основные сценарии работы системы, роли пользователей, архитектура микросервисов и механизм авторизации на основе OpenID Connect и JWT.

3 Технологическая часть

3.1 Выбор операционной системы

Согласно требованиям технического задания, разрабатываемая система должна запускаться в среде оркестрации Kubernetes. Для узлов кластера была выбрана операционная система Linux, дистрибутив Ubuntu. **Ubuntu** [1] является распространенным серверным дистрибутивом, поддерживает контейнеризацию, имеет развитую экосистему пакетов и широко применяется при развертывании микросервисных систем.

3.2 Выбор СУБД

В качестве СУБД была выбрана **PostgreSQL** [2]. Данная СУБД используется сервисами автомобилей, бронирований, платежей, авторизации и статистики. Каждый сервис работает со своей логической базой данных:

- база `cars` хранит автомобили;
- база `rentals` хранит бронирования;
- база `payments` хранит платежи;
- база `identity` хранит учетные записи пользователей;
- база `statistics` хранит события системы.

PostgreSQL была выбрана из-за поддержки транзакций, надежности, удобной работы с UUID и распространенности в микросервисных приложениях.

3.3 Выбор языка разработки и backend-фреймворка

Для разработки backend-сервисов был выбран **Python** [3]. Данный язык позволяет быстро создавать прототипы, имеет развитую экосистему библиотек для работы с HTTP API, базами данных, JWT и Kafka.

В качестве backend-фреймворка был выбран **FastAPI** [4]. Он подходит для реализации микросервисов по следующим причинам:

- высокая скорость разработки REST API;

- поддержка декларативной валидации входных данных;
- автоматическая генерация OpenAPI-описания;
- удобная интеграция с middleware авторизации;
- поддержка асинхронной обработки запросов.

3.4 Выбор фреймворка пользовательского интерфейса

Для реализации пользовательского интерфейса был выбран **React** [5]. Интерфейс реализован как Single Page Application, отдаваемое ui-service. React позволяет строить интерактивные страницы без перезагрузки, динамически обновлять список автомобилей, бронирований и административные отчеты.

3.5 Асинхронное взаимодействие

Для асинхронного взаимодействия между сервисами используется **Apache Kafka** [6]. Gateway-service отправляет события о действиях пользователя в Kafka topic `car-rental-events`. Statistics-service читает эти события, сохраняет их в своей базе данных и строит отчет для администратора.

Через Kafka передаются следующие типы событий:

- регистрация пользователя;
- создание бронирования;
- завершение бронирования;
- отмена бронирования.

3.6 Реализация микросервисов

Система состоит из следующих сервисов.

3.6.1 UI-service

UI-service отдает HTML-страницу с React-приложением. Пользовательский интерфейс содержит:

- экран входа и регистрации;
- список автомобилей;
- выбор дат аренды;
- расчет стоимости;
- список бронирований пользователя;
- административную панель со списком пользователей и статистикой.

3.6.2 Identity-service

Identity-service выполняет роль собственного Identity Provider [7]. Сервис реализует Authorization Code Flow, выдачу JWT, JWKs, регистрацию пользователей, создание пользователей администратором и ролевую модель Admin/User.

При старте сервиса автоматически создается администратор. Новые пользователи, созданные через регистрацию или администратором, получают роль User.

3.6.3 Cars-service

Cars-service отвечает за хранение и выдачу данных об автомобилях. Для каждого автомобиля хранится марка, модель, регистрационный номер, мощность, цена за день, тип и признак доступности. Сервис предоставляет методы получения списка автомобилей, получения автомобиля по идентификатору и изменения доступности автомобиля.

3.6.4 Rental-service

Rental-service отвечает за бронирования. Сервис хранит идентификатор бронирования, пользователя, автомобиль, платеж, даты начала и окончания аренды, а также статус. Возможные статусы: IN_PROGRESS, FINISHED, CANCELED.

3.6.5 Payment-service

Payment-service отвечает за платежи. При создании бронирования gateway-service создает платеж на рассчитанную сумму. Платеж может находиться в статусе PAID или CANCELED.

3.6.6 Statistics-service

Statistics-service получает события из Kafka, сохраняет их в БД и предоставляет административные методы получения сводки и списка событий. Доступ к данным статистики разрешен только пользователю с ролью Admin.

3.6.7 Gateway-service

Gateway-service является сервисом агрегирования запросов. Он принимает запросы от UI и внешних клиентов, валидирует JWT, вызывает внутренние сервисы и формирует агрегированные ответы. Например, при получении списка бронирований gateway-service добавляет к бронированию сведения об автомобиле и платеже.

3.7 Авторизация и межсервисное взаимодействие

Все сервисы используют общий middleware для проверки JWT. Токен передается в заголовке `Authorization: Bearer`. Валидация выполняется по JWKs. При ошибке авторизации сервис возвращает HTTP 401.

Gateway-service пробрасывает JWT во внутренние сервисы. Таким образом, межсервисные запросы также защищены token-based авторизацией, а пользователь определяется из claims JWT.

3.8 Обработка ошибок и деградация

В системе предусмотрена обработка ошибок зависимых сервисов. Если сервис статистики или Kafka временно недоступны, основная пользовательская операция не блокируется. При недоступности платежного сервиса gateway-service использует механизм деградации для сохранения работоспособности сценариев бронирования и отмены. Ошибки сервисов преобразуются в корректные HTTP-ответы с соответствующими статусами.

3.9 Логирование

Во всех сервисах добавлено логирование входящих HTTP-запросов: метод, путь, статус ответа и время выполнения. Gateway-service также логирует ключевые бизнес-события и отправляет их в Kafka для последующей обработки statistics-service.

3.10 Развертывание

Все сервисы контейнеризуются и разворачиваются в Kubernetes [8]. Для развертывания используется универсальный Helm chart `service-chart` [9]. Отличия между сервисами задаются через values-файлы:

- `identity-service.yaml`;
- `ui-service.yaml`;
- `cars-service.yaml`;
- `rental-service.yaml`;
- `payment-service.yaml`;
- `statistics-service.yaml`;
- `gateway-service.yaml`;
- `kafka.yaml`.

PostgreSQL разворачивается отдельным Helm chart `postgres-chart`, который создает базу данных и job инициализации схем.

Для публикации сервисов наружу используется Ingress Controller `ingress-nginx`. Наружу публикуются только:

- `/` — пользовательский интерфейс;
- `/api` и `/manage` — gateway-service;
- `/oauth` и `/.well-known` — identity-service.

3.11 CI/CD

Сборка и развертывание выполняются через GitHub Actions. Pipeline выполняет следующие шаги:

1. запуск unit-тестов;
2. сборка Docker-образов сервисов;
3. публикация Docker-образов;
4. развертывание PostgreSQL, Kafka и сервисов в Kubernetes через Helm;
5. запуск Newman-тестов для проверки Gateway API.

3.12 Высокоуровневый дизайн пользовательского интерфейса

Пользовательский интерфейс системы представляет собой веб-интерфейс, доступный через браузер. Основные страницы системы:

- страница входа и регистрации;
- страница списка автомобилей;
- страница бронирований пользователя;
- страница администрирования;
- страница статистики.

Примеры страниц приведены на рисунках 3.1-3.8.

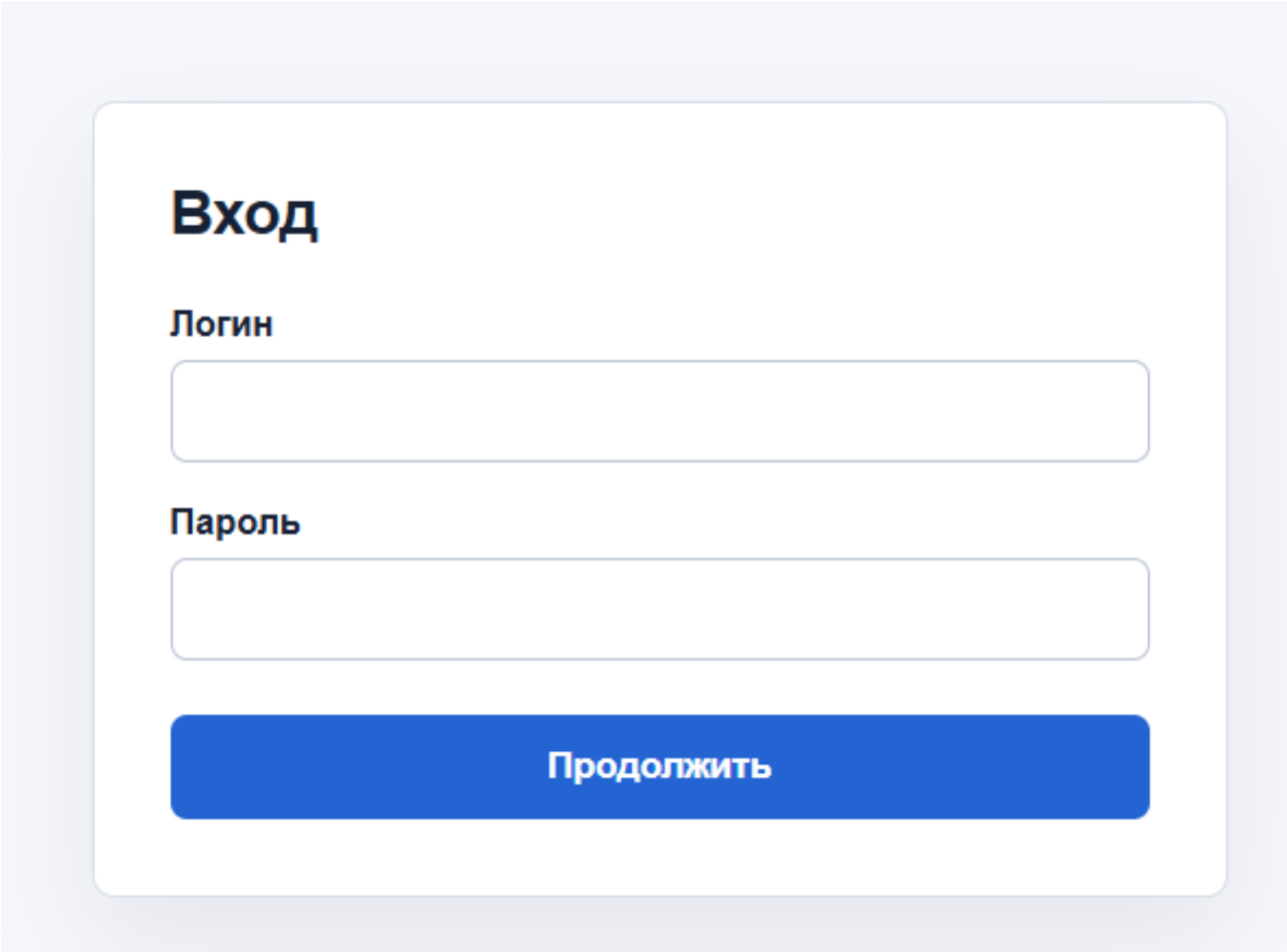


Рисунок 3.1 – Страница входа

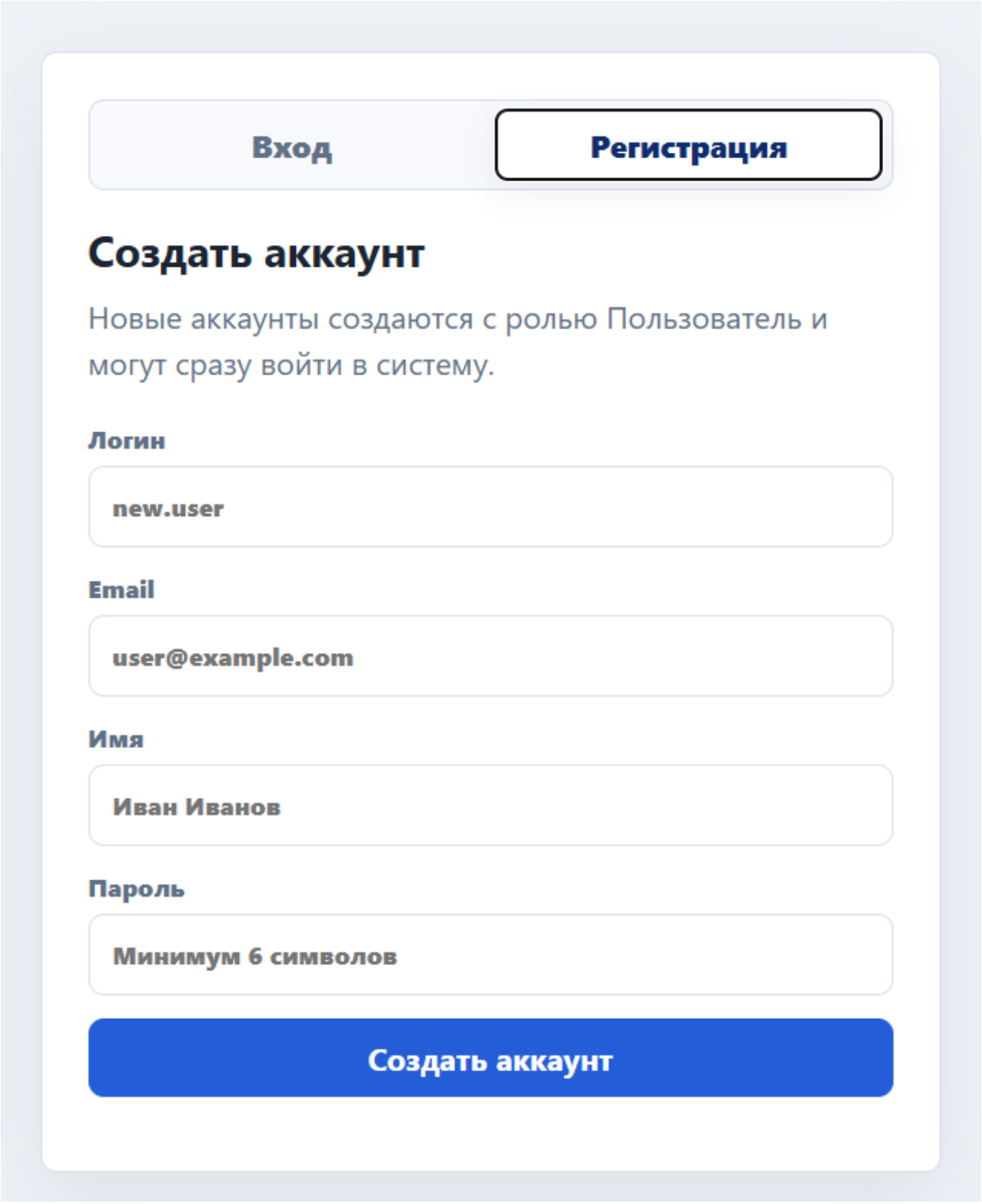


Рисунок 3.2 – Страница регистрации

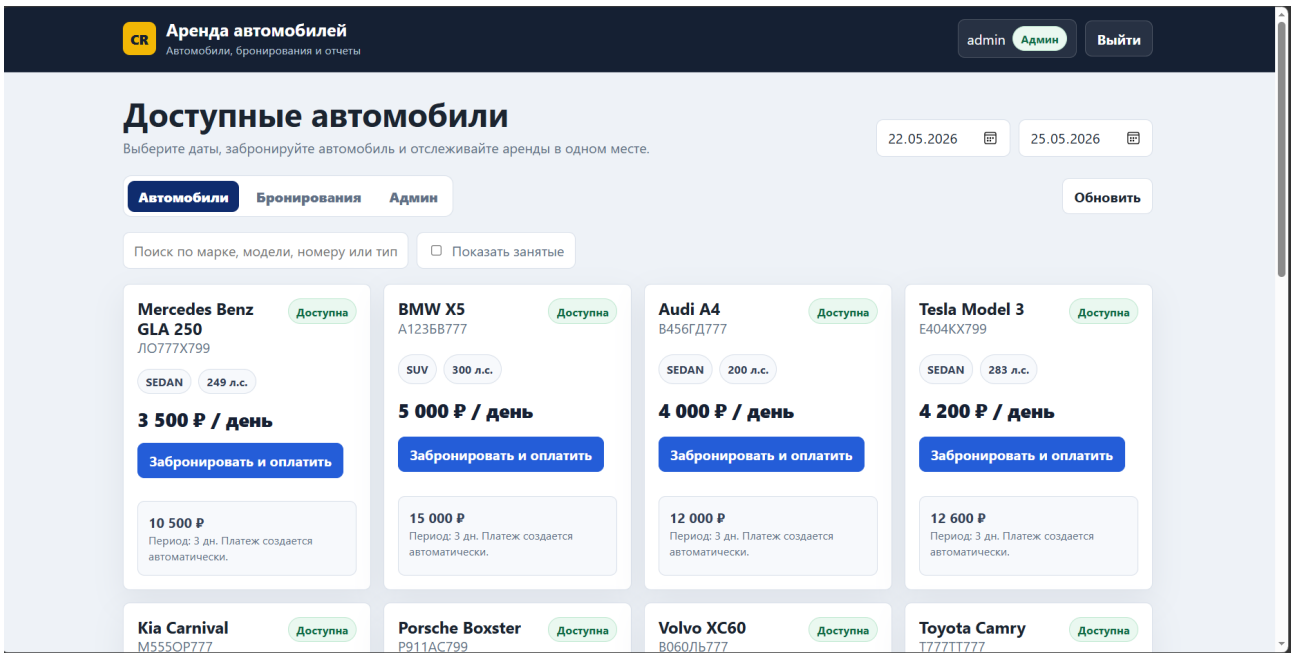


Рисунок 3.3 – Страница списка автомобилей

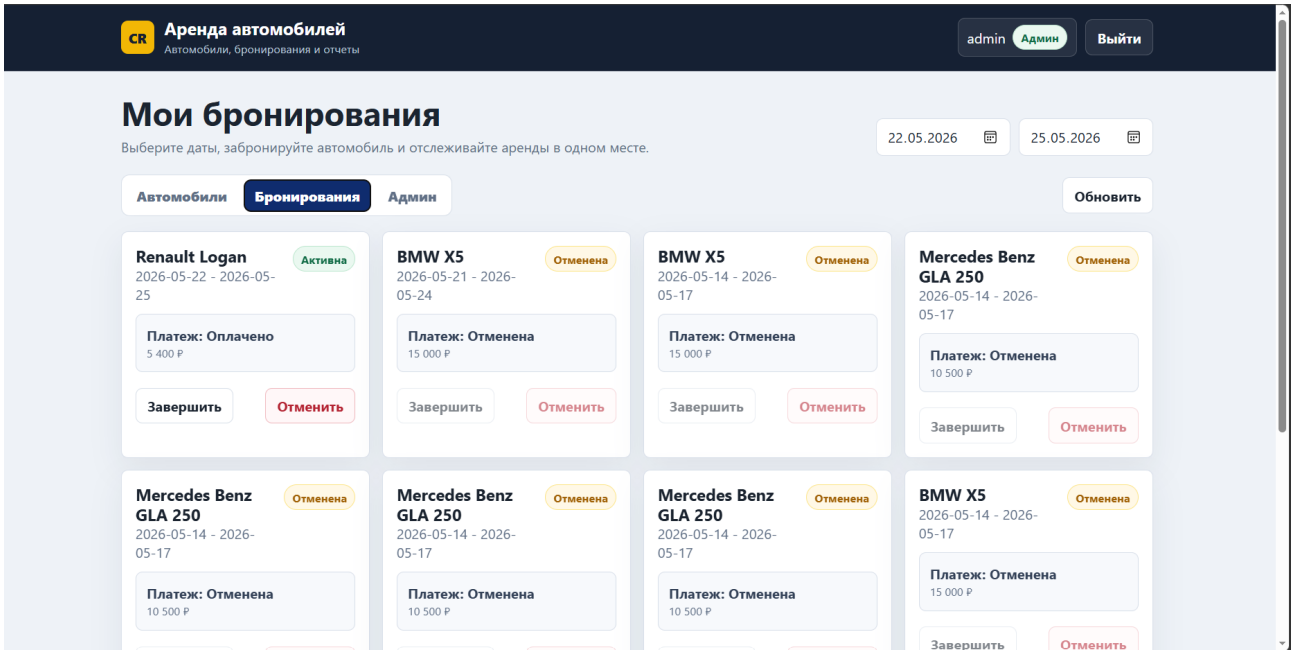


Рисунок 3.4 – Страница бронирования автомобиля

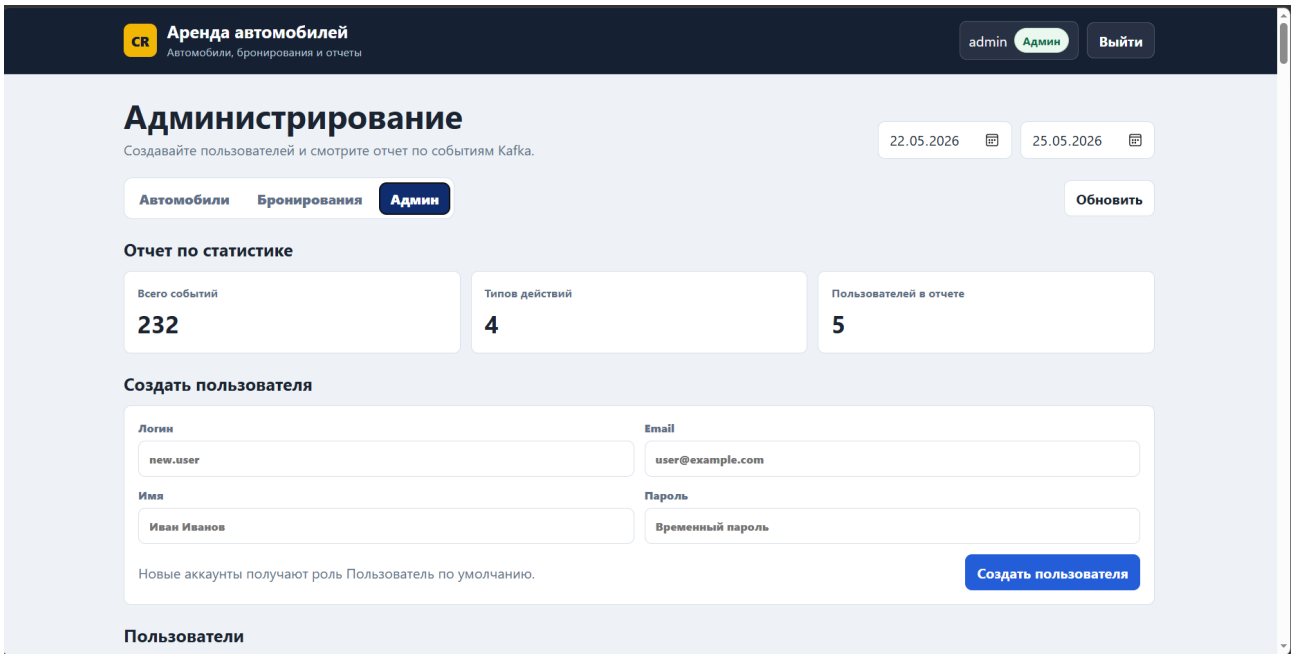


Рисунок 3.5 – Страница бронирований пользователя

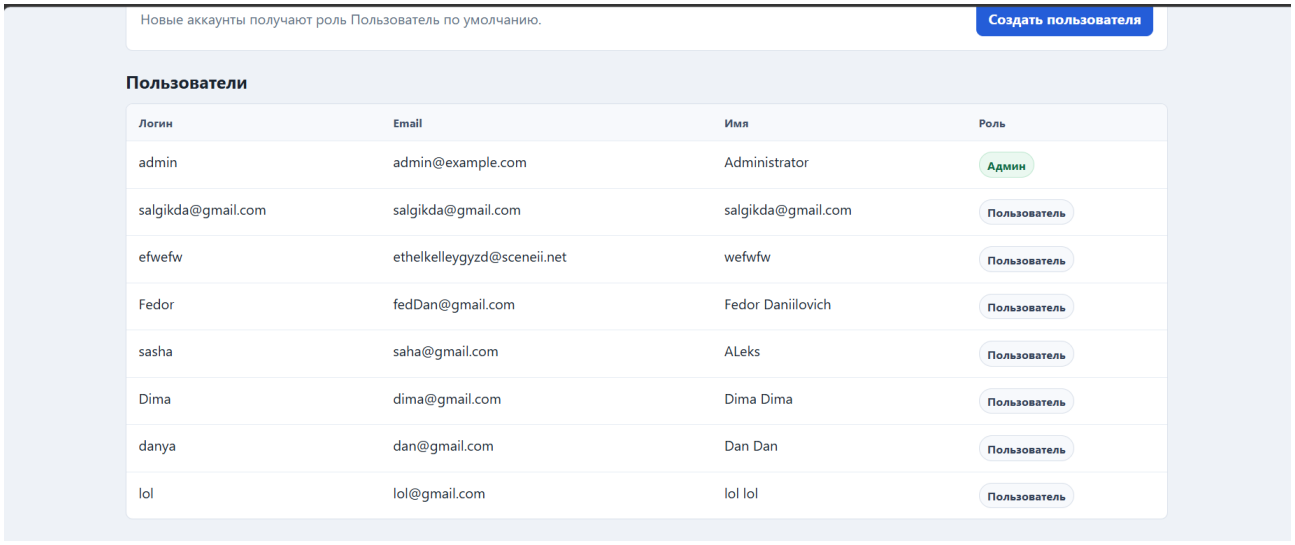
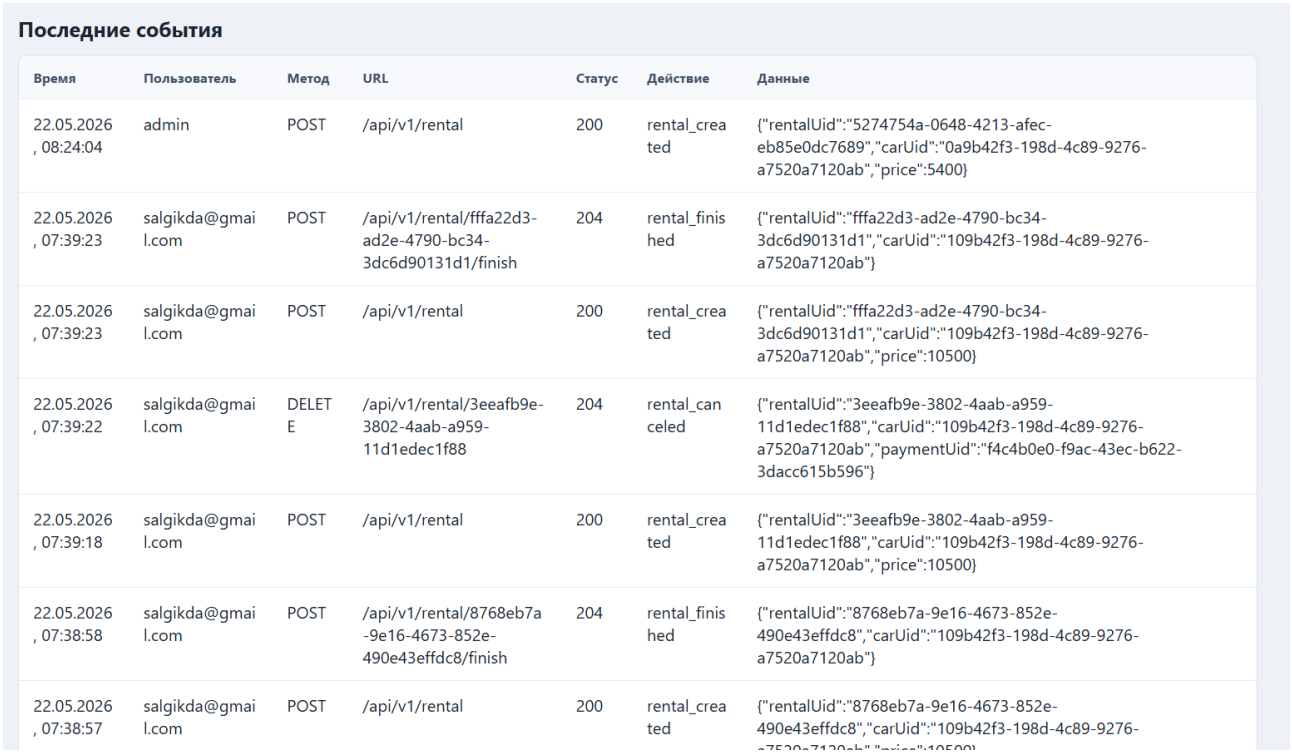


Рисунок 3.6 – Страница администрирования пользователей



Последние события

Время	Пользователь	Метод	URL	Статус	Действие	Данные
22.05.2026, 08:24:04	admin	POST	/api/v1/rental	200	rental_created	{"rentalUid":"5274754a-0648-4213-afec-eb85e0dc7689","carUid":"0a9b42f3-198d-4c89-9276-a7520a7120ab","price":5400}
22.05.2026, 07:39:23	salgikda@gmail.com	POST	/api/v1/rental/fffa22d3-ad2e-4790-bc34-3dc6d90131d1/finish	204	rental_finished	{"rentalUid":"fffa22d3-ad2e-4790-bc34-3dc6d90131d1","carUid":"109b42f3-198d-4c89-9276-a7520a7120ab"}
22.05.2026, 07:39:23	salgikda@gmail.com	POST	/api/v1/rental	200	rental_created	{"rentalUid":"fffa22d3-ad2e-4790-bc34-3dc6d90131d1","carUid":"109b42f3-198d-4c89-9276-a7520a7120ab","price":10500}
22.05.2026, 07:39:22	salgikda@gmail.com	DELETE	/api/v1/rental/3eeafb9e-3802-4aab-a959-11d1edec1f88	204	rental_canceled	{"rentalUid":"3eeafb9e-3802-4aab-a959-11d1edec1f88","carUid":"109b42f3-198d-4c89-9276-a7520a7120ab","paymentUid":"f4c4b0e0-f9ac-43ec-b622-3dacc615b596"}
22.05.2026, 07:39:18	salgikda@gmail.com	POST	/api/v1/rental	200	rental_created	{"rentalUid":"3eeafb9e-3802-4aab-a959-11d1edec1f88","carUid":"109b42f3-198d-4c89-9276-a7520a7120ab","price":10500}
22.05.2026, 07:38:58	salgikda@gmail.com	POST	/api/v1/rental/8768eb7a-9e16-4673-852e-490e43effdc8/finish	204	rental_finished	{"rentalUid":"8768eb7a-9e16-4673-852e-490e43effdc8","carUid":"109b42f3-198d-4c89-9276-a7520a7120ab"}
22.05.2026, 07:38:57	salgikda@gmail.com	POST	/api/v1/rental	200	rental_created	{"rentalUid":"8768eb7a-9e16-4673-852e-490e43effdc8","carUid":"109b42f3-198d-4c89-9276-

Рисунок 3.7 – Страница статистики

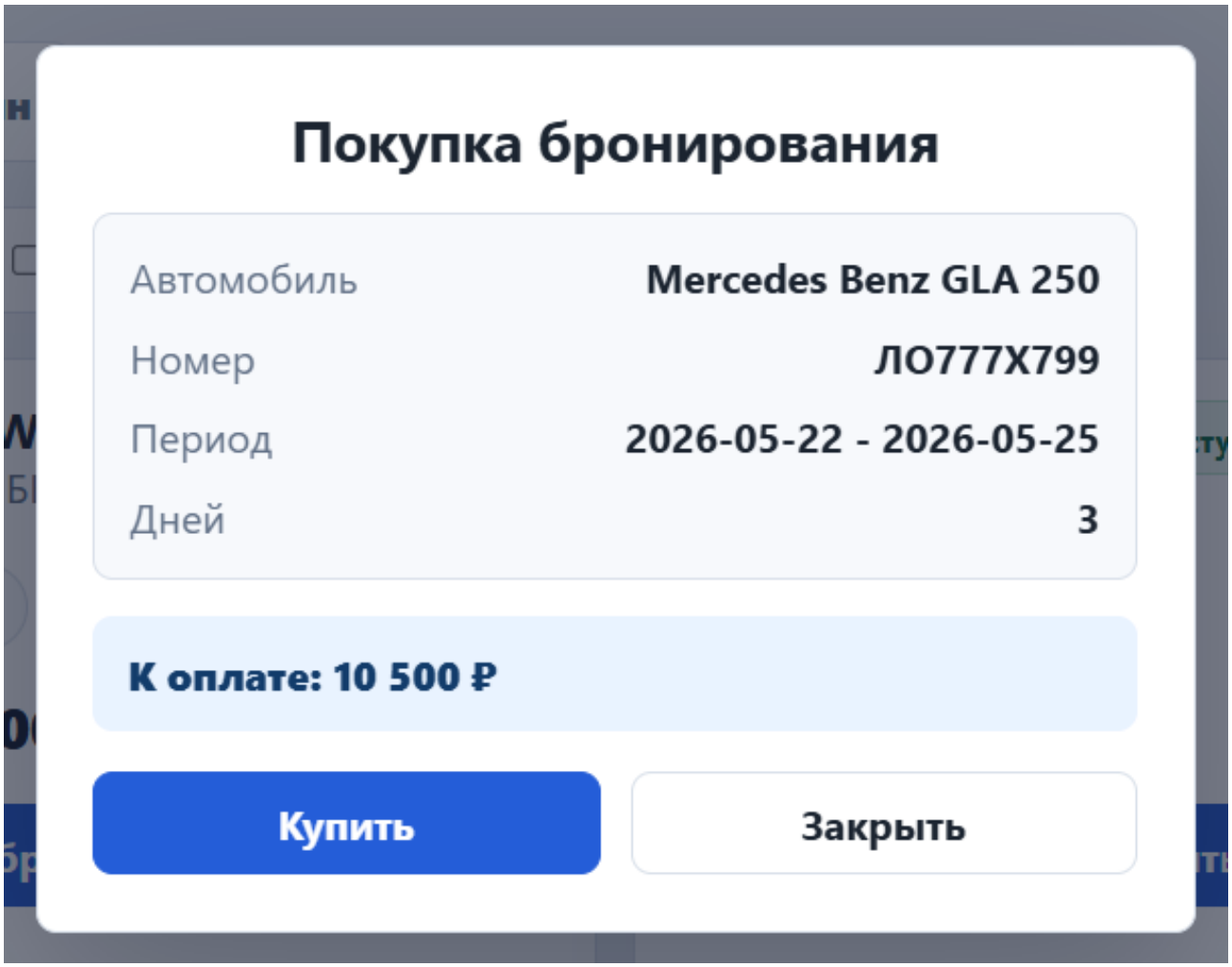


Рисунок 3.8 – Окно подтверждения покупки бронирования

3.13 Вывод

В технологической части были выбраны основные технологии реализации, описаны микросервисы системы, механизм авторизации, обработка ошибок, логирование, развертывание в Kubernetes и CI/CD pipeline.

ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы был разработан прототип системы бронирования автомобилей на выбранные даты. Система реализована в виде набора микросервисов, каждый из которых отвечает за собственную область: пользовательский интерфейс, авторизацию и учетные записи, автомобили, бронирования, платежи, статистику и агрегацию запросов.

Для backend-сервисов использован FastAPI, для пользовательского интерфейса — React. В качестве СУБД используется PostgreSQL. Для асинхронной передачи событий и последующего построения статистики используется Apache Kafka. Авторизация реализована через собственный Identity Provider с поддержкой OpenID Connect, JWT и JWKs. Пользователи разделены на роли User и Admin, а доступ к административной статистике ограничен ролью Admin.

Система разворачивается в Kubernetes с использованием Helm charts. Для публикации наружу настроен Ingress Controller. Сборка Docker-образов, развертывание и запуск Newman-тестов выполняются через GitHub Actions.

Таким образом, были решены следующие задачи:

- описана предметная область системы бронирования автомобилей;
- сформулированы функциональные требования к системе;
- спроектирована микросервисная архитектура;
- реализован пользовательский веб-интерфейс;
- реализован собственный Identity Provider с OpenID Connect;
- реализованы сервисы автомобилей, бронирования, платежей, статистики и gateway;
- настроена передача событий через Kafka;
- реализована token-based авторизация межсервисных запросов;
- подготовлено развертывание системы в Kubernetes через Helm;
- настроен CI/CD pipeline.

Цель курсовой работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ubuntu [Электронный ресурс]. — Режим доступа URL: https://ubuntu.com (Дата обращения: 16.05.2026).
2. PostgreSQL [Электронный ресурс]. — Режим доступа URL: https://www.postgresql.org (Дата обращения: 16.05.2026).
3. Python [Электронный ресурс]. — Режим доступа URL: https://www.python.org (Дата обращения: 16.05.2026).
4. FastAPI [Электронный ресурс]. — Режим доступа URL: https://fastapi.tiangolo.com (Дата обращения: 16.05.2026).
5. React [Электронный ресурс]. — Режим доступа URL: https://react.dev (Дата обращения: 16.05.2026).
6. Apache Kafka [Электронный ресурс]. — Режим доступа URL: https://kafka.apache.org (Дата обращения: 16.05.2026).
7. Identity Provider [Электронный ресурс]. — Режим доступа URL: https://www.okta.com/identity-101/why-your-company-needs-an-identity-provider/ (Дата обращения: 16.05.2026).
8. Kubernetes [Электронный ресурс]. — Режим доступа URL: https://kubernetes.io (Дата обращения: 16.05.2026).
9. Helm [Электронный ресурс]. — Режим доступа URL: https://helm.sh (Дата обращения: 16.05.2026).